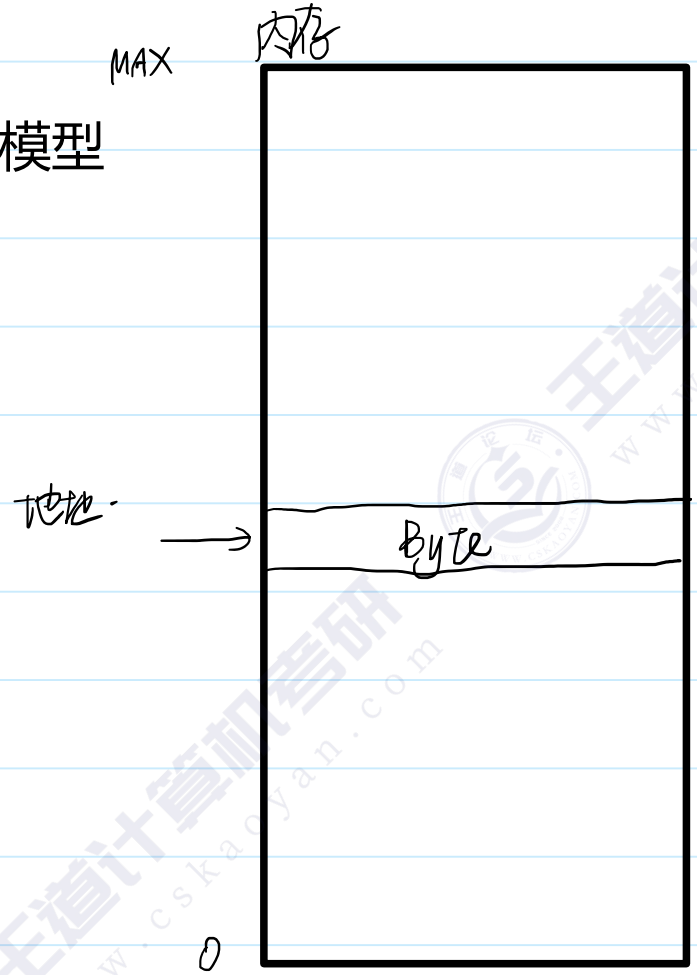
地址、指针和指针变量

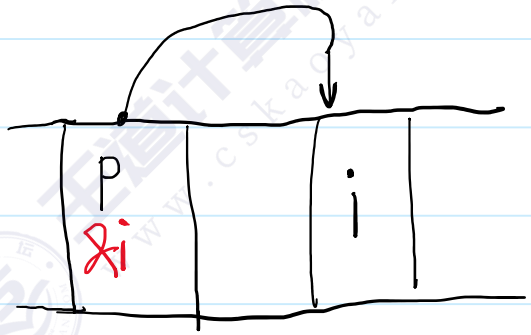
内存模型



我们想根据地址来访问内存中的数据

指针就是地址

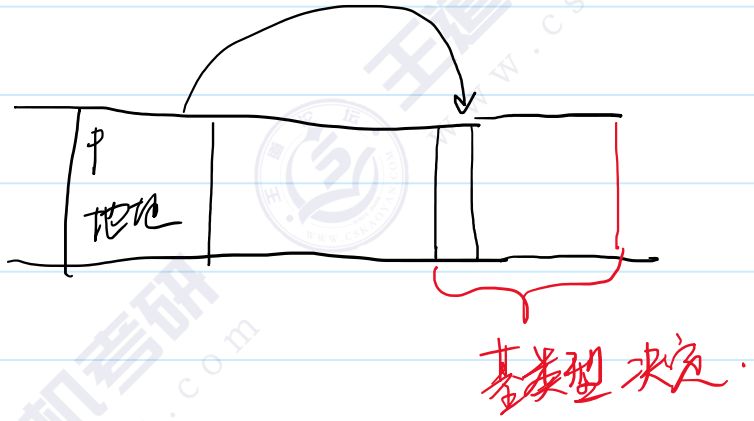
指针变量是一个变量，变量的内存里面存储了一个指针



指针变量的定义和初始化

前提：先给目标申请内存，然后再来创建指针变量

基类型 *指针变量名



```c
int main() {
    int i = 10;
    //int* p; // 不推荐这种写法
    //int* p, q; //q是一个int
    //int *p, *q; //推荐将*和变量名写在一起

    //int *p;
    // &运算符出现在非定义语句里面，表示取地址的意思
    //p = &i; //p里面存储的地址值是i的地址 p指向i
    // 对指针变量赋值可以修改指针的指向

    int *p = &i; //定义一个指针变量p，初始化为指向i
    return 0;    已用时间 <= 1ms
}
```

```
0x000000D8F575FBD4   0a 00 00 00
0x000000D8F575FBD8   cc cc cc cc
0x000000D8F575FBDC   cc cc cc cc
0x000000D8F575FBE0   cc cc cc cc
0x000000D8F575FBE4   cc cc cc cc
0x000000D8F575FBE8   cc cc cc cc
0x000000D8F575FBEC   cc cc cc cc
0x000000D8F575FBF0   cc cc cc cc
0x000000D8F575FBF4   cc cc cc cc
0x000000D8F575FBF8   d4 fb 75 f5
0x000000D8F575FBFC   d8 00 00 00
```

间接访问

```
int i
int *p = &i; //定义一个指针变量p，初始化为指向i

printf("*p = %d\n", *p);
// *运算符在定义语句里面，用来说明创建指针变量
// *运算符出现非定义语句里面，解引用/间接访问
// 根据指针变量里面存储的地址和指针变量的基类型，去访问目标的内容
*p = 11;
printf("i = %d\n", i);
return 0;
}
```

```
Microsoft Visual Studio 调试控制台
*p = 10
i = 11
```

流程：

1、给目标去申请内存

2、准备一个指针变量，通过赋值or初始化的方式，让指针变量去指向目标

3、使用解引用运算符去间接访问目标

两种用途

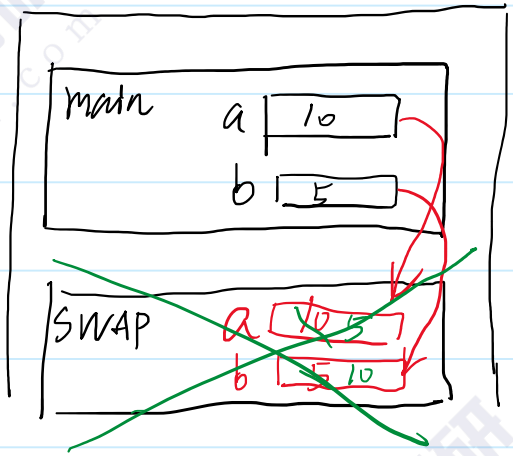
1、传递 指针和函数配合

2、偏移 指针和数组配合

指针的传递

```c
#include <stdio.h>
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("swap a = %d, b = %d\n", a, b);
}
int main() {
    int a = 10, b = 5;
    swap(a, b);
    printf("main a = %d, b = %d\n", a, b);
}
```

```
Microsoft Visual Studio 调试控制台
swap a = 5, b = 10
main a = 10, b = 5
```

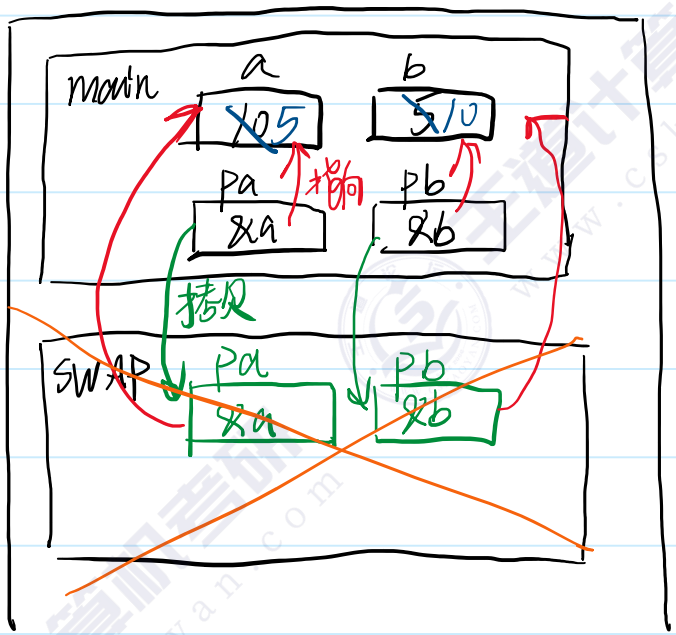


由于值传递机制，被调函数中没有办法修改主调函数栈帧里面的内容

* 解引用间接访问

```c
void swap(int* pa, int* pb) {
    int temp = *pa;
    *pa = *pb;
    *pb = temp;
    printf("swap *pa = %d, *pb = %d\n", *pa, *pb);
}
int main() {
    int a = 10, b = 5;
    int *pa, *pb;
    pa = &a;
    pb = &b;
    swap(pa, pb);
    printf("main a = %d, b = %d\n", a, b);
}
```

```
Microsoft Visual Studio 调试控制台
swap *pa = 5, *pb = 10
main a = 5, b = 10
```



指针的传递

指针的传递可以用来在被调函数中去修改主调函数栈帧的数据

1、主调方先给被修改的数据申请内存
2、被调方需要用数据的指针作为参数
3、被调方需要用解引用运算符去间接访问内存，修改内容

指针的偏移

对指针变量做加减法（只能加减整数）

int *p p和p+1有什么关系?

```c
int main() {
  int i = 10;
  double d = 3.14;
  int* pi = &i;
  double* pd = &d;
  printf("pi = %p, pi+1 = %p\n", pi, pi+1);//指针可以用%p作为占位符
  printf("pd = %p, pd+1 = %p\n", pd, pd+1);
  return 0;
}
```

```
选择 Microsoft Visual Studio 调试控制台
pi = 000000FDDE99F594, pi+1 = 000000FDDE99F598
pd = 000000FDDE99F5B8, pd+1 = 000000FDDE99F5C0
```

p+n的结果 = p里面存储的地址 + n*sizeof(基类型)

和口诀算符很像.

```
int arr[] = { 1,2,3,4,5 };
int* p = &arr[0];
for (int i = 0; i < 5; ++i) {
  printf("i = %d, p+i = %p, &arr[i] = %p\n",
    i, p+i, &arr[i]);
  printf("i = %d, *(p+i) = %d, arr[i] = %d\n",
    i, *(p+i), arr[i]);
}
r
```

Microsoft Visual Studio 调试控制台

```
i = 0, p+i = 000000BDD875FC08, &arr[i] = 000000BDD875FC08
i = 0, *(p+i) = 1, arr[i] = 1
i = 1, p+i = 000000BDD875FC0C, &arr[i] = 000000BDD875FC0C
i = 1, *(p+i) = 2, arr[i] = 2
i = 2, p+i = 000000BDD875FC10, &arr[i] = 000000BDD875FC10
i = 2, *(p+i) = 3, arr[i] = 3
i = 3, p+i = 000000BDD875FC14, &arr[i] = 000000BDD875FC14
i = 3, *(p+i) = 4, arr[i] = 4
i = 4, p+i = 000000BDD875FC18, &arr[i] = 000000BDD875FC18
i = 4, *(p+i) = 5, arr[i] = 5
```

指针和数组更加有意思的关联

```
int arr[] = { 1,2,3,4,5 };
//int* p = &arr[0];
int* p = arr; //数组名赋值给指针变量
for (int i = 0; i < 5; ++i) {
  printf("i = %d, p+i = %p, &arr[i] = %p\n",
    i, p+i, &arr[i]);
  printf("i = %d, *(p+i) = %d, arr[i] = %d\n",
    i, *(p+i), arr[i]);
  printf("i = %d, p[i] = %d, *(arr+i) = %d\n",
    i, p[i], *(arr+i));
}
re
```

Microsoft Visual Studio 调试控制台

```
i = 0, p+i = 0000005DBFAFFC28, &arr[i] = 0000005DBFAFFC28
i = 0, *(p+i) = 1, arr[i] = 1
i = 0, p[i] = 1, *(arr+i) = 1
i = 1, p+i = 0000005DBFAFFC2C, &arr[i] = 0000005DBFAFFC2C
i = 1, *(p+i) = 2, arr[i] = 2
i = 1, p[i] = 2, *(arr+i) = 2
i = 2, p+i = 0000005DBFAFFC30, &arr[i] = 0000005DBFAFFC30
i = 2, *(p+i) = 3, arr[i] = 3
i = 2, p[i] = 3, *(arr+i) = 3
i = 3, p+i = 0000005DBFAFFC34, &arr[i] = 0000005DBFAFFC34
i = 3, *(p+i) = 4, arr[i] = 4
i = 3, p[i] = 4, *(arr+i) = 4
i = 4, p+i = 0000005DBFAFFC38, &arr[i] = 0000005DBFAFFC38
i = 4, *(p+i) = 5, arr[i] = 5
i = 4, p[i] = 5, *(arr+i) = 5
```

1、数组不是指针，指针也不是数组

2、数组的数组名可以赋值给一个指针变量；也可以做加减法 ---> 数组的数组名可以退化成&arr[0]

3、指针可以使用[]运算符

p[i] 等价于 *(p+i)

```
printf("sizeof(p) = %d, sizeof(arr) = %d\n",
  sizeof(p), sizeof(arr));
```

```
sizeof(p) = 8, sizeof(arr) = 20
```

使用指针的原则

指针非常容易犯错

原则：我们必须先确保目标的存在，再考虑指向它的指针

```c
#include <stdio.h>
int *func() {
    int arr[] = { 1,2,3 };
    for (int i = 0; i < 3; ++i) {
        printf("func arr[i] = %d\n", arr[i]);
    }
    return arr; //数组会退化成指针
}
int main() {
    int* p = func();
    for (int i = 0; i < 3; ++i) {
        printf("main p[i] = %d\n", p[i]);
    }
    return 0;
}
```

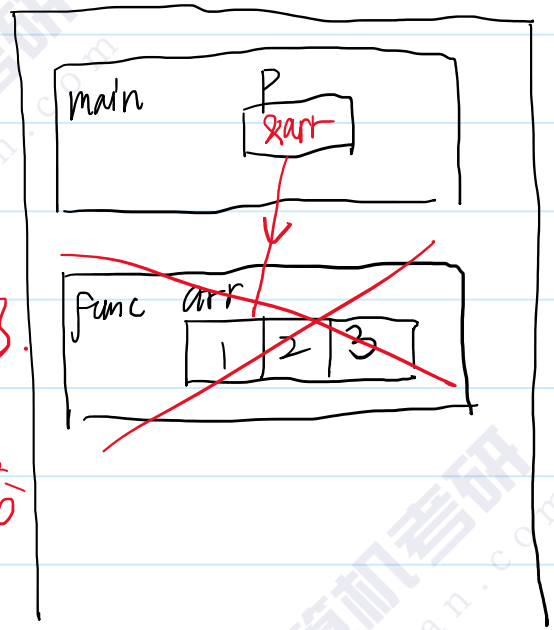
func返回时，栈被销毁了。

无效内容

```
C:\Users\liaozs\source\repos\C语言督学营\x64\Debug\023
func arr[i] = 1
func arr[i] = 2
func arr[i] = 3
main p[i] = 1
main p[i] = -858993460
main p[i] = -858993460
```

未找到相关问题

值



结论：

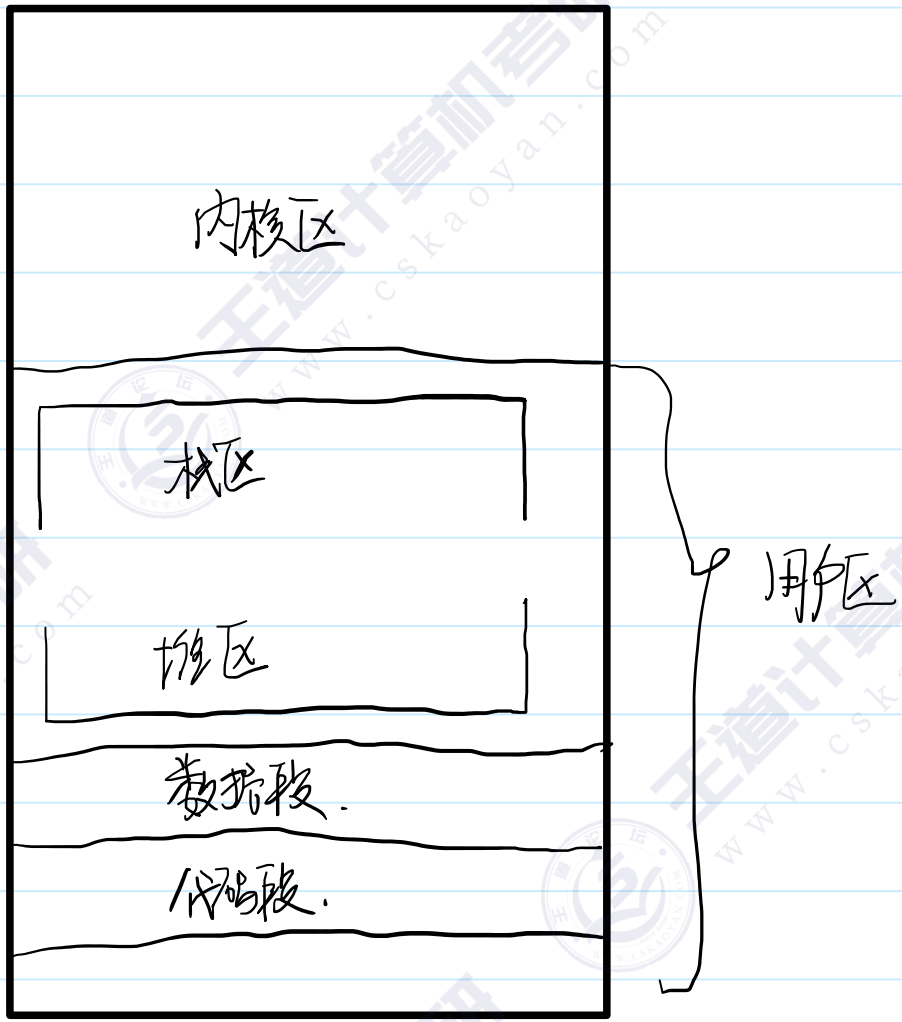
函数的返回值不能返回指向自己局部变量的指针

堆空间

内存布局

malloc 申请内存

free 释放内存



栈区：函数调用的时候申请内存

函数返回的时候释放内存

数据段：放全局变量

程序运行的时候申请内存

程序结束的时候释放内存

堆区：

由用户自己决定什么时候申请内存，什么时候释放内存

malloc

malloc

语法:

```
#include <stdlib.h>
void *malloc( size_t size );
```

功能： 函数指向一个大小为*size*的空间，如果错误发生返回NULL。

1、添加一个#include<stdlib.h>

2、函数名是malloc

参数是一个size_t类型 无符号的8字节整数--- > 描述申请空间的大小

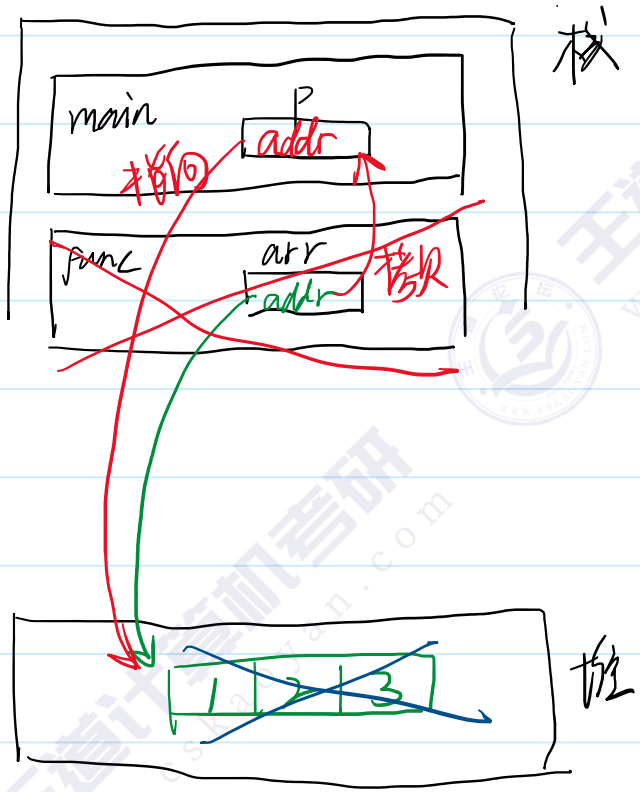
返回值是一个void *(基类型还没有确定的指针，void *类型的指针变量在解引用和偏移之前必须先强转成其他类型) ---> 描述申请空间的首地址

动态数组

```c
int* func() {
    int* arr = (int*)malloc(3 * sizeof(int));
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 3;
    for (int i = 0; i < 3; ++i) {
        printf("func arr[i] = %d\n", arr[i]);
    }
    return arr;
}
int main() {
    int* p = func();
    for (int i = 0; i < 3; ++i) {
        printf("main p[i] = %d\n", p[i]);
    }
    free(p); //避免内存泄漏
}
```

选择 Microsoft Visual Studio 调试控制台

```
func arr[i] = 1
func arr[i] = 2
func arr[i] = 3
main p[i] = 1
main p[i] = 2
main p[i] = 3
```



堆空间的长度是可以在运行的时候确定的

空指针和野指针

```c
void func(int* pi) {
  *pi = 100;
}
int main() {
  //int* p;
  //p = (int*)0x12345678;
  ////这是一个野指针
  //*p = 100;
  //printf("*p = %d\n", *p);

  //int* p = NULL; //NULL底层就是0 --> 空指针
  //int i = 101;
  //p = &i;
  //*p = 100;

  int* p = NULL;
  int i = 100;
  p = &i;
  func(p);
  return 0;
}
```

C风格的字符串

字符串由一个或者多个字符组成的字符序列

C语言没有原生的字符串类型

C语言的字符串是基于字符数组 char数组

C字符串 = 字符数组 + 在字符串的有效内容之后，要额外加一个ASCII码为0的终止符('\0')作为结尾

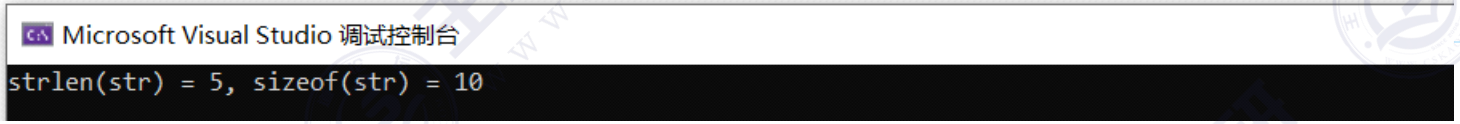
和字符串相关的函数

strlen

```
#include <string.h>

size_t strlen( char *str );
```

```
char str[10] = "hello"; // 数组长度 大于等于 有效长度+1 即可
printf("strlen(str) = %d, sizeof(str) = %d\n", strlen(str), sizeof(str));
return 0;
}
```



strcpy 拷贝字符串

```
char *strcpy( char *to, const char *from );
```

```
char to[10];
char from[10] = "hello";
// to = from; 数组是不能赋值的
strcpy(to, from);
printf("to = %s\n", to);
return 0;
}
```

strcpy是有风险的

```
strcpy(to, from);
```

from里面的内容有可能比to占用的空间更大

```
char to[5];
char from[10] = "hello";
// to = from; 数组是不能赋值的
strcpy(to, from);
printf("to = %s\n", to);
return 0;
}
```

strcmp 比较字符串的大小

字典序

"a" < "abandon" < "b"

strcmp

语法:

```
#include <string.h>
int strcmp( const char *str1, const char *str2 );
```

功能：比较字符串*str1* and *str2*, 返回值如下:

返回值	解释
less than 0	str1 is less than str2
equal to 0	str1 is equal to str2
greater than 0	str1 is greater than str2

```
char str1[] = "back";
char str2[] = "backward";
char str3[] = "back";
printf("str1 vs str2 = %d\n", strcmp(str1, str2));
printf("str2 vs str1 = %d\n", strcmp(str2, str1));
printf("str1 vs str3 = %d\n", strcmp(str1, str3));
return 0;
```

Microsoft Visual Studio 调试控制台

```
str1 vs str2 = -1
str2 vs str1 = 1
str1 vs str3 = 0
```

strcat 字符串拼接

"how" "ever" --> "however"

strcat

语法:

```
#include <string.h>
char *strcat( char *str1, const char *str2 );
```

```
char str1[20] = "how";
char str2[] = "ever";
strcat(str1, str2);
printf("str1 = %s\n", str1);
```

```
Microsoft Visual Studio 调试控制台
str1 = however
```

strcat也是不安全的 ---> str1参数里面没有说明长度 --> 拼接之后长度有可能超过str1申请的内存范围

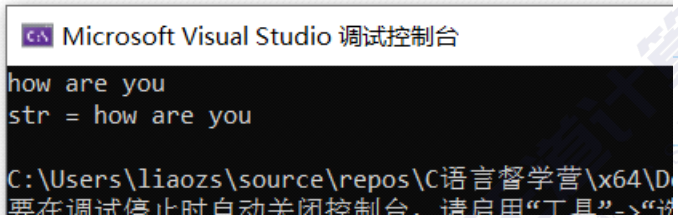
从标准输入中读取字符串

scanf %s ---> 读取一个单词 ---> 以空白字符为边界，换行、空格、制表符

fgets ---> 读取一行句子 ---> 以换行为边界

```
//char str[20];
//scanf("%s", str); //str是一个数组 退化成指针
//printf("str = %s\n", str);

char str[20];
fgets(str, 20, stdin);//从stdin中读取一行到str，最大长度是20
int idx = strlen(str) - 1;
if (str[idx] == '\n') {
    str[idx] = '\0';
}
printf("str = %s\n", str);
```



scanf和fgets的安全性

scanf的参数里面没有长度信息 --> 不安全

```
char str[4];
scanf("%s", str);
printf("str = %s\n", str);
return 0;
}
```

fgets是否安全?

安全

```
char str[10];
fgets(str, 10, stdin);
printf("str = %s\n", str);
return 0;
```